Analysis of the Classical No-Prop Algorithm for Supervised Learning

Project Presentation II (Final Presentation)

Aispur P. Santiago

School of Mathematical and Computational Sciences
Yachay Tech University, Urcuqui, Ecuador
santiago.aispur@yachaytech.edu.ec

2026-05-04

Outline

1. Motivation and Objective
2. Formal Definition
3. Implementation
4. Mathematical Analysis
5. Empirical Results
6. Conclusion and Resources

Why Look Beyond Backpropagation?

- Backpropagation is effective but computationally intensive:
  - Requires forward + backward passes.
  - Stores intermediate activations (memory cost).
  - Training latency increases with model size.
- Motivation: alternative training paradigms with improved efficiency.
- Focus: Classical No-Prop (train only the output layer; hidden layers are frozen).

Project Goal

Main objective

Provide a formal algorithmic analysis of Classical No-Prop and validate it empirically.

- Formal definition + pseudocode.
- Time/space complexity (best/avg/worst).
- C++ implementation and empirical evaluation (HTML phishing detection).

Notation and Setup

- Dataset: $D = \{(x_i, y_i)\}_{i=1}^{S}$, $x_i \in \mathbb{R}^N$, $y_i \in \mathbb{R}^M$.
- One-hidden-layer MLP:
  - Frozen hidden layer: $W_1 \in \mathbb{R}^{N \times H}$, $b_1 \in \mathbb{R}^H$.
  - Trainable output layer: $W_2 \in \mathbb{R}^{H \times M}$, $b_2 \in \mathbb{R}^M$.
- Activation: $\phi(\cdot)$ (e.g., ReLU).
- Learning rate: η.

No-Prop Forward Pass and Update Rule

Forward pass

$$h = \phi(x^T W_1 + b_1), \qquad \hat{y} = hW_2 + b_2$$

Error and LMS (Delta) update

$$e = y - \hat{y}$$

$$W_2 \leftarrow W_2 + \eta \cdot h^T e, \qquad b_2 \leftarrow b_2 + \eta \cdot e$$

- Key difference: no gradient propagation to W_1, b_1.

Pseudocode (Classical No-Prop)

Algorithm 1 Classical No-Prop Training

Input: $D = \{(x_i, y_i)\}_{i=1}^{S}$, learning rate η, epochs E.
Initialize $W_1 \sim \mathcal{N}(0, 1)$, $b_1 \leftarrow 0$ (freeze).
Initialize $W_2 \leftarrow 0$, $b_2 \leftarrow 0$ (trainable).
for epoch $= 1$ to E **do**
    for each (x, y) in D **do**
        $h \leftarrow \phi(x^T W_1 + b_1)$
        $\hat{y} \leftarrow hW_2 + b_2$
        $e \leftarrow y - \hat{y}$
        $W_2 \leftarrow W_2 + \eta \cdot h^T e$
        $b_2 \leftarrow b_2 + \eta \cdot e$

Implementation Details (C++ / Eigen)

- Implemented from scratch in C++ for fair timing measurements.
- Linear algebra via **Eigen**.
- Two comparable models:
  - `NoPropMLP`: frozen hidden layer, LMS updates only on output layer.
  - `BackpropMLP`: standard MLP trained with backpropagation (updates all weights).

Dataset and Preprocessing

- Dataset: `HTML_Top13_Features.csv`
  - 19,997 samples, 13 numerical features.
  - Balanced binary labels (phishing vs benign).
- Split: train 60%, validation 20%, test 20%.
- Standardization (z-score) computed on training only (avoid leakage).

Task

Binary classification: $\mathbb{R}^{13} \to \{0, 1\}$.

Design Paradigm

- Classical No-Prop follows an online learning paradigm:
  - process one sample, update immediately.
- Interpretation:
  - Frozen hidden layer $\Rightarrow$ fixed non-linear feature mapping.
  - Output layer $\Rightarrow$ adaptive linear model on those features.
- Optimization of output layer with MSE is convex w.r.t. (W_2, b_2) for fixed features.

Time Complexity (One Hidden Layer)

Let S samples, E epochs, input N, hidden H, output M (binary: $M = 1$).

No-Prop per sample

- Hidden activation: $x^T W_1$ is $O(NH)$.
- Output + update: $O(HM)$.

Dominant term: $O(NH)$.

Total training time

$$T_{\text{No-Prop}} = O(E \cdot S \cdot NH)$$

$$T_{\text{BP}} = O(E \cdot S \cdot NH)$$

Asymptotically equal, but backprop has a larger constant factor (backward pass).

Space Complexity

No-Prop

- Store W_1: $O(NH)$, b_1: $O(H)$
- Store W_2: $O(HM)$, b_2: $O(M)$
- Activations (online): $O(H)$

Dominant: $O(NH)$

Backprop (online)

- Same parameter storage: $O(NH)$
- Needs activations for backward pass (still $O(H)$ in online 1-layer case)

Also $O(NH)$; advantage grows with depth.

Experimental Setup

- Train for 50 epochs.
- Vary hidden width: $H \in \{16, 32, 64, 128, 256\}$.
- Measure:
  - Test accuracy
  - Total wall-clock training time

Results Summary (Table)

Hidden Width	No-Prop Acc.	BP Acc.	No-Prop Time (s)	BP Time (s)
16	0.5078	0.5000	15.52	27.60
32	0.5560	0.5000	10.51	16.73
64	0.5783	0.5000	10.92	16.42
128	0.5475	0.5000	11.22	17.45
256	0.5000	0.5000	13.42	19.87

Accuracy and Training-Time Discussion

- Backprop baseline stayed at $\approx 50\%$ (chance) $\Rightarrow$ suggests poor hyperparameters / non-convergence in this setup.
- No-Prop achieved best observed accuracy at $H = 64$:

$$\text{Acc}_{\text{max}} \approx 57.83\%$$

- No-Prop was consistently faster across all widths.
- Interpretation: same asymptotic order, but BP has higher constant factor due to backward pass.

Figures (Insert Your Plots)

Place your generated plots in the same directory and uncomment the lines.

Training time vs width	Log-log time vs width

Conclusion

- Classical No-Prop avoids the backward pass by freezing hidden layers.
- Complexity (one hidden layer, online):
  - Time: $O(ESNH)$ (same order as BP, lower constants).
  - Space: $O(NH)$ (advantage increases with depth).
- Empirically: No-Prop training time consistently lower; accuracy is task/initialization dependent.

Project Resources

- Overleaf (LaTeX Source):
  https://www.overleaf.com/read/hspcrqqffspx#d52cef
- GitHub (C++ Implementation):
  https://github.com/icepursantiago/Analysis-of-Algorithms

Q & A

Questions?